Adaboost Model. (1995, 2000)

El corazon del modelo es una minimizacion greedy de la perdida exponencial

$$L = \sum_i e^{(-y_i \cdot F(x_i))} \qquad \text{(Ec. 1)}$$

Dado un dataset, el objetivo es encontrar los T stumps $h_1, h_2 \ldots h_T$ y sus pesos $\alpha_1, \alpha_2, \ldots, \alpha_T$ que minimicen la perdida exponencial de manera iterativa

Optimizacion greedy: en el paso 1 eliges la mejor decision disponible dado que no sabes nada del futuro. En el paso 2 eliges la mejor opcion disponible dado que lo ya decidiste en el paso 1 (que ya esta fijo, no lo vas a cambiar). Y asi en cada paso mirar solo el siguiente movimiento optimo, sin planificar a largo plazo.

Nota: la palabra greedy en ingles significa codicioso, porque en cada momento agarras lo mejor disponible ahora, sin pensar que te conviene a largo plazo

y_i solo toma 2 valores $\quad y_i \in \{-1, +1\}$ (y_i pertenece al conjunto)

$F(x_i) = \alpha_1 h_1(x) + \alpha_2 h_2(x) + \ldots +$ (suma ponderada de stumps) (Ec. 2)

Ponde:

$h_K(x) \in \{-1, 1\}$, es la prediccion del stump

y:

$\alpha_K > 0$ peso del stump K

α_K es siempre positivo y se calcula con la formula alpha (α)

$$\alpha = 0.5 \cdot \ln\left[\frac{1 - err}{err}\right] \qquad \text{(Ec. 3)}$$

La ecuacion 2 nos dice que cada stump K vota +1 (tarjeta) o -1 (efectivo) y su voto se multiplica por α. El total $F(x)$ es la votacion ponderada.

Siendo: $F_{t-1}(x) = \alpha_1 h_1(x) + \alpha_2 h_2(x) + \ldots + \alpha_{t-1} h_{t-1}(x)$ (Ec. 4)

$F(x) = F_{t-1}x + \alpha_t h_t(x)$ (Ec 5.))

Reemplazo (5) en (1):

$$L = e^{(-y_i \cdot F(x_i))} = e^{(-y_i \cdot [F_{t-1}(x_i) + \alpha \cdot h_t(x)]}$$

Nota: uso propiedad $e^{[a+b]} = e^a \cdot e^b$

$$L = \underbrace{e^{(-y_i \cdot F_{t-1}(x_i))}}_{w_i} \cdot e^{(-y_i \cdot \alpha \cdot h_t(x_i))}$$

Nota: La perdida exponencial es "amigable" con el greedy. Por su forma $e^{a+b} = e^a \cdot e^b$, el problema se Factoriza limpiamente: optimizar el termino actual no requiere conocer los terminos futuros, solo los pasados (que ya estan Fijos)

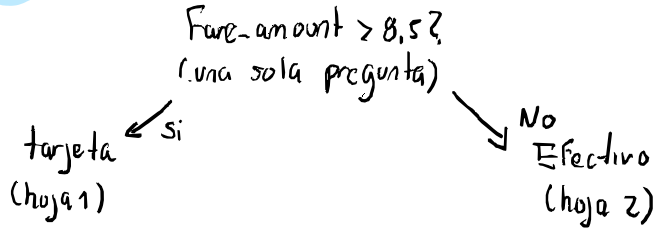
Dado que F_{t-1} es fijo (no optimizamos sobre el) w_i es constante, minimizamos:

$$L = \sum_i w_i \cdot e^{(-y_i \cdot \alpha h_t(x_i))} \quad \text{(Ec 6.)}$$

Nota: la razon para usar el conjunto $\{-1, 1\}$ para y_i y $h(x_i)$ es por la propiedad: si la prediccion es correcta $\rightarrow y \cdot h(x) = +1$ y si es incorrecta $\rightarrow y \cdot h(x) = -1$

Tanto si $y_i = 1$ y predijo $(h(t)) +1$ $(1 \cdot 1 = 1)$ como si $y_i = -1$ y predijo -1 $(-1 \cdot -1 = 1)$ siempre da $+1$ cuando acierta y siempre -1 cuando Falla. Esto convierte $y \cdot F(x)$ en una medida unificada de "que tan de acuerdo esta el modelo con la realidad. De este modo el $e^{(-y_i \cdot F(x_i))}$ se vuelve una perdida que crece cuando el modelo se equivoca y se reduce cuando acierta, todo en una expresion limpia

Abordemos la formula alpha (α): $\alpha = 0.5 \cdot \ln[(1-err)/err]$, pero que es el err? que es el termino err? err se calcula luego de entrenar el stump, evaluamos en datos de entrenamiento. Un stump se ve asi:



$$err = \sum_i w_i \cdot I[y_i \neq h_t(x_i)] \quad \text{(Ec 7)}$$

$0.10(1) + 0.10(1) + 0.10(1) = 0.30$

Donde:

- w_i es el peso actual de la observacion i
- $I[y_i \neq h_t(x_i)]$ Funcion indicadora: vale 1 si el stump se equivoca en esa observacion y 0 si acierta.

En resumen: err es la suma de los pesos de las observaciones que el stump clasifico mal. Veamos un ejemplo numerico paso a paso

Ejemplo: Dataset de 10 viajes.

Iteracion 1: pesos iniciales. En la iteracion 1 todos los pesos son iguales

↳ Inicializacion $w_i = \frac{1}{N} \quad \forall i$ → 1/10

Si el dataset tiene 10 viajes, cada uno arranca con $w = 0.10$. Si hay 1000 viajes cada uno arranca con $w = 0.001$. La idea es: "no se nada todavia, todas las observaciones cuentan igual y esos pesos por construccion suman $(N \cdot (1/N) = 1)$

En la iteracion 1 el objetivo es encontrar el (h_i, α_i) que minimice la perdida en este momento sin ningun modelo previo

Todos los viajes empiezan iguales

$w^{(1)} = [0.10, 0.10, 0.10, 0.10, 0.10, 0.10, 0.10, 0.10, 0.10, 0.10]$ Suma = 1.0

Entrenas el primer stump. Supongamos que falla en los viajes 3, 5 y 10

Calculamos el err segun Ec 7.

$$err = 0.10 + 0.10 + 0.10 = 0.30$$

Calculo el alpha segun Ec 3: $\alpha_1 = 0.5 \cdot \ln\left[\frac{1 - 0.30}{0.30}\right] = 0.5 \cdot \ln(2.33) = 0.424$

Actualizacion iteracion 2: Aqui esta la magia del Adaboost. Despues de entrenar el stump t, calcular su err y su α, los pesos se actualizan multiplicativa/

$$w_i^{nuevo} = w_i^{viejo} \cdot e^{(-y_i \cdot \alpha_t \cdot h_t(x_i))}$$

$F_1 = \alpha_1 \cdot h_1$ ya esta fijo (no lo voy a tocar mas). Me ocupo de encontrar el (h_2, α_2) que minimice la perdida añadiendo el termino F_1

Recordemos las viajes acertados: 1, 2, 4, 6, 7, 8, 9

Recordemos la funcion de perdida: $L = \sum_i w_i \cdot e^{(-y_i \cdot \alpha \cdot h_t(x_i))}$

Cuando acierta y_i y $h_t(x_i)$ toman los mismos valores: +1 y +1 ó -1 y -1 y en este caso cuando acierta el termino $-y_i \cdot \alpha \cdot h_t(x_i))$ es negativo

$w_{nuevo} = \underbrace{0.10}_{w_i \text{ inicial}} \cdot e^{(-0,424)} = 0,0654$

Viajes Fallados: 3, 5, 10

Cuando la prediccion falla y_i y $h_t(x_i)$ toman valores distintos +1 y -1 y en consecuencia el termino $-y_i \cdot \alpha \cdot h_t(x_i))$ es positivo.

$w_{nuevo} = 0.10 \cdot e^{(+0,424)} = 0,1528$

Pesos antes de normalizar:

$w_{sn} = [0.0654, 0.0654, 0.1528, 0.0654, 0.1528, 0.0654, 0.0654, 0.0654, 0.0654, 0.1528]$

$w_{sn} = 0,916$

Normalizamos dividiendo todo por 0.916

{ acertadas: 0.0654/0.916 = 0.0714
Fallados: 0.1528/0,916 = 0.1668

Iteracion 2: pesos resultantes

$w^{(2)} = [0.0714, 0.0714, 0.1668, 0.0714, 0.1668, 0.0714, 0.0714, 0.0714, 0.0714, 0.1668]$

suma = 1.0

Ya se tienen pesos no uniformes: los viajes 3, 5 y 10 (los que el stump falló) ahora pesan mas del doble que el resto. El stump 2 los "va a ver con lupa". Despues de muchas iteraciones, los pesos pueden quedar muy desiguales: las observaciones que han sido sistematicamente mal clasificadas acumulan peso mientras que las "faciles" lo van perdiendo.

Iteracion 3: $F_2 = \alpha_1 h_1 + \alpha_2 h_2$ ya esta FIJO. El objetivo es encontrar el (h_3, α_3) que minimice la funcion de perdida

Resumen del ciclo completo:

- Iteracion 1: pesos uniformes $(1/N)$
- Entrenar stump: calcular err y alpha (α)
- Actualizar pesos $(w_i) \rightarrow$ subir los fallos y bajar los aciertos
- Renormalizar (que sumen 1)
- Iteracion 2: pesos (w_i) no uniformes.
- Entrenar stump (con sample weight) $\rightarrow$ calcular err y α y asi sucesivamente

Aprendimos que los w_i no son datos de entrada ni los asignamos; el algoritmo los crea y los va modificando solo. Empiezan iguales, y en cada iteracion se redistribuyen para concentrar la atencion en las observaciones dificiles. Ese es el "Adaptative" del nombre Adaboost: los pesos se adaptan iteracion a iteracion a las dificultades que el ensamble va encontrando. Cuando el error tiende a cero, el alpha tiende a infinito.

err	(1-err)/err	alpha	wi(acierta)	wi(Falla)
0.30	2.33	0.424	0.0654	0.1528
0.10	9.0	1.098	0.333	3

Cuando el error tiende a cero, el alpha (α) tiende a infinito. La formula recompensa cada vez mas el stump por cada decima que el error reduce. Es la propiedad que hace que adaboost le de "voz mas fuerte" a los mejores clasificadores y "voz debil" a los mediocres automaticamente.

La idea clave para llevarse hasta ahora:

Adaboost no resuelve el problema gigante de encontrar los 100 mejores 100 stumps a la vez. Resuelve 100 problemas pequeños en secuencia: en cada paso busca el siguiente stump y su peso optimos para reducir la perdida un poquito mas, asumiendo que todo lo anterior esta fijo y no se va a tocar.

Es la version matematica de "construir incrementalmente": cada nueva pieza se añade sobre la torre, nunca se desmonta lo ya construido. Para Boosting esta sencilla estrategia produce modelos sorprendentemente potentes.

Ejemplo concreto con tres stumps

Supongamos que despues de 3 iteraciones Adaboost ha entrenado estos stumps:

Stump	Regla	Alpha
h_1	Si fare_amount > 8.5 → +1 (tarjeta), sino −1	$\alpha_1 = 0.38$
h_2	Si trip_distance > 2 → +1 (tarjeta) sino −1	$\alpha_2 = 0,53$
h_3	Si airport == 1 → +1 (tarjeta) sino −1	$\alpha_3 = 0.75$

Veamos como calcular F(x) para 3 viajes distintos:

Viaje A → un viaje claramente "tipo tarjeta"

Datos: fare = 25 , distance = 8 Km , airport = 1

Cada stump vota:

h_1: 25 > 8,5 → vota +1

h_2: 8 > 2 → vota +1

h_3: airport = 1 → vota 1

$F(A) = 0.38(+1) + 0,53(+1) + 0,75(+1) = +1,66$ → positivo y grande, los tres stumps coinciden en tarjeta. El ensamble es seguro

Viaje B – un viaje claramente tipo efectivo

Datos: fare = 4 , distance = 0,5 km , airport = 0

h_1: $4 < 8.5$ → vota −1

h_2: $0.5 < 2$ → vota −1

h_3: airport = 0 → vota −1

$F(B) = 0.38(-1) + 0,53(-1) + 0,75(-1) = -1,66$ → negativo y grande. Los 3 stumps coinciden en efectivo. Ensamble seguro

Viaje C. Un viaje ambiguo

datos: fare = 12, distance = 3.Km airport = 0

h_1: 12 > 8.5 → vota +1

h_2: 3 > 2 → vota +1

h_3: airport = 0 vota −1

$F(C) = 0{,}38(1) + 0{,}53(1) + 0{,}75(-1) = +0{,}16$ → positivo pero pequeño. Dos stumps dicen tarjeta y uno dice efectivo, y el que dice efectivo (h_3) tiene el alpha mas grande, asi que cancela los otros dos. El ensamble se inclina por Tarjeta pero con muy poca confianza.

Lo que aprendemos del ejemplo:

Viaje	F(x)	Prediccion (signo)	confianza (magnitud)
A	+1,66	tarjeta	Muy alta
B	−1,66	efectivo	Muy alta
C	+0,16	tarjeta	Muy baja

El signo de F dice que predice el ensamble. Positivo: tarjeta, negativo: efectivo. La magnitud de F dice que tan seguro esta. $|F| = 1{,}66$ es ensamble convencido, $|F| = 0{,}16$ es ensamble dubitativo.

F puede tomar cualquier valor real. Su rango depende de cuantas stumps haya y de que tan grandes sean sus alpha. Si tienes 100 stumps con alphas alrededor de 0,5, F puede ir aproximadamente de −50 a +50

Porque importa la perdida?: Recordando que la perdida es $e^{(-y \cdot F(x))}$, ahora puede verse que la magnitud importa

Para el viaje A (tarjeta real, $y=+1$, $F=1,66$): pérdida $= e^{(-1 \cdot 1,66)}$ $= e^{-1,66} = 0,19$. Pérdida pequeña, el modelo lo tiene bien clasificado y con confianza

Si el viaje C fuera realmente efectivo ($y=-1$, pero $F=+0,16$, predice tarjeta): pérdida $= e^{(-(-1)\cdot 0,16)} = e^{0,16} = 1.17$. Pérdida moderada, Fallo pero el ensamble tampoco estaba muy seguro

Si el viaje A fuera realmente efectivo ($y=-1$, pero $F=+0,16$, predice tarjeta) pérdida: $e^{(-(-1)\cdot 1,66)} = e^{1,66} = 5,26$. Pérdida enorme. El modelo equivoco y encima de eso estaba muy convencido - lo peor que puede pasar

Por eso la pérdida exponencial penaliza tan fuertemente los errores confiados; gracias a que F tiene una magnitud, el sistema "sabe la diferencia entre equivocarse con duda y equivocarse con seguridad. Y al iterar Adaboost se concentra obsesivamente en arreglar esos errores grandes

Resumen: